

WEEK -5

Session-9:

Implement a producer-consumer communication system using anonymous pipes where the parent process generates data and the child process consumes it. Measure the communication efficiency.

```
GNU nano 8.7.1 producer_consumer.c
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <string.h>
#include <limits.h>

void change_directory(char *path) {
    char oldpwd[PATH_MAX];
    char newpwd[PATH_MAX];

    if (getcwd(oldpwd, sizeof(oldpwd)) == NULL) {
        perror("getcwd");
        return;
    }

    if (path == NULL || strcmp(path, "~") == 0) {
        path = getenv("HOME");
    }

    if (strcmp(path, "-") == 0) {
        path = getenv("OLDPWD");

        if (path == NULL) {
            printf("OLDPWD not set\n");
            return;
        }

        printf("%s\n", path);
    }

    if (chdir(path) == -1) {
        perror("cd");
        return;
    }

    if (getcwd(newpwd, sizeof(newpwd)) == NULL) {
        perror("getcwd");
        return;
    }

    setenv("OLDPWD", oldpwd, 1);
    setenv("PWD", newpwd, 1);
}

int main() {
    char command[100];

    while (1) {
```

[Read 66 lines]

OUTPUT:

```
pavani@DESKTOP-380LEIN:~$ gcc cd_shell.c -o cd_shell
pavani@DESKTOP-380LEIN:~$ ls
5      buffer.c      file1.txt  helloCopy.java  ossp      recall.c  state.c
OSSP   cd_shell       file2.txt  main.java       process   script.sh test.txt
a.txt  cd_shell.c       hello     myfifo         process.c shortcut  txt
b.txt  command_executor hello.c    myfile.txt      producer_consumer.c snap      week
buffer command_executor.c hello.sh   os-lab         recall       state     week3
pavani@DESKTOP-380LEIN:~$ ./cd_shell
shell> pwd
Unknown command
shell> cd /tmp
shell> cd -
/home/pavani
shell> exit
```

Report: The cd command was implemented to change the current working directory using chdir().

The program validates the given path and displays an error for invalid paths.

PWD and OLDPWD are maintained to track the current and previous directories.

Question 2: TO Identify Built-ins, Create Dispatch Table, Execute In- Process Commands, Handle Invalid Commands, Maintain State, Test Dispatch Logic

```

GNU nano 8.7.1 dispatch_shell.c *
char input[200];

while (1) {
    printf("shell> ");

    fgets(input, sizeof(input), stdin);
    input[strcspn(input, "\n")] = '\0';

    char *command = strtok(input, " ");
    char *arg = strtok(NULL, " ");

    if (command == NULL)
        continue;

    int found = 0;

    for (int i = 0; i < 4; i++) {
        if (strcmp(command, builtins[i].name) == 0) {
            builtins[i].function(arg);
            found = 1;
            break;
        }
    }

    if (found)
        continue;

    pid_t pid = fork();

    if (pid == 0) {
        if (arg != NULL)
            execlp(command, command, arg, NULL);
        else
            execlp(command, command, NULL);

        perror("Command");
        exit(1);
    }
    else if (pid > 0) {
        wait(NULL);
    }
    else {
        perror("fork");
    }
}

return 0;
}

```

OUTPUT:

```

pavani@DESKTOP-380LEIN:~$ gcc dispatch_shell.c -o dispatch_shell
pavani@DESKTOP-380LEIN:~$ ls
5      cd_shell      file1.txt      main.java      process.c      snap      week3
OSSP   cd_shell.c        file2.txt      myfifo         producer_consumer.c  state
a.txt  command_executor  hello         myfile.txt     recall         state.c
b.txt  command_executor.c hello.c        os-lab         recall.c       test.txt
buffer dispatch_shell    hello.sh      ossp           script.sh      txt
buffer.c dispatch_shell.c  helloCopy.java process         shortcut       week
pavani@DESKTOP-380LEIN:~$ ./dispatch_shell
shell> pwd
/home/pavani
shell> cd /tmp
shell> pwd
/tmp
shell> export NAME=Pavani
shell> ls
snap-private-tmp
systemd-private-288bf2e24bf64d1ea01e694d6470ef22-chroney.service-UZdyn0
systemd-private-288bf2e24bf64d1ea01e694d6470ef22-systemd-logind.service-qKM0nk
shell> exit

```

Report: A dispatch table was created to identify and execute built-in commands such as cd, pwd, export, and exit.

Function pointers were used to connect commands with their respective functions.

External commands were handled using fork() and exec().

Session : 10

Question 3: To Retrieve Current Directory, Display path, Process Exit Requests, Cleanup Resources, Save State, Verify Correct Behavior

```
GNU nano 8.7.1                                pwd_exit_shell.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>

void pwd_command() {
    char path[1024];

    if (getcwd(path, sizeof(path)) != NULL)
        printf("%s\n", path);
    else
        perror("pwd");
}

void exit_command() {
    printf("Exiting shell...\n");
    printf("Resources cleaned successfully.\n");
    exit(0);
}

int main() {
    char input[100];

    while (1) {
        printf("shell> ");

        fgets(input, sizeof(input), stdin);
        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "pwd") == 0) {
            pwd_command();
        }
        else if (strcmp(input, "exit") == 0) {
            exit_command();
        }
        else {
            printf("Unknown command\n");
        }
    }

    return 0;
}
```

OUTPUT:

```
shell> exit
pavani@DESKTOP-380LEIN:~$ nano pwd_exit_shell.c
pavani@DESKTOP-380LEIN:~$ gcc pwd_exit_shell.c -o pwd_exit_shell
pavani@DESKTOP-380LEIN:~$ ./pwd_exit_shell
shell> pwd
/home/pavani
shell> hello
Unknown command
shell> exit
Exiting shell...
Resources cleaned successfully.
pavani@DESKTOP-380LEIN:~$
```

Report: The pwd command was implemented using getcwd() to display the current working directory.

The exit command terminates the shell after performing the required cleanup.

The program was tested to verify correct directory display and termination.

Question 4: To Parse Export Syntax, Update Environment Variables, Validate Names, Handle Existing Variables, Export to Child Processes, Test Functionality

```
*equal = '\0';

char *name = input;
char *value = equal + 1;

if (!valid_name(name)) {
    printf("export: invalid variable name\n");
    return;
}

if (setenv(name, value, 1) == -1) {
    perror("export");
    return;
}

printf("Exported %s=%s\n", name, value);
}

int main() {
    char input[200];

    while (1) {
        printf("shell> ");

        fgets(input, sizeof(input), stdin);
        input[strcspn(input, "\n")] = '\0';

        if (strncmp(input, "export ", 7) == 0) {
            export_command(input + 7);
        }
        else if (strcmp(input, "exit") == 0) {
            break;
        }
        else if (strncmp(input, "print ", 6) == 0) {
            char *value = getenv(input + 6);

            if (value != NULL)
                printf("%s\n", value);
            else
                printf("Variable not found\n");
        }
        else {
            printf("Unknown command\n");
        }
    }

    return 0;
}
```

OUTPUT:

```
pavani@DESKTOP-380LEIN:~$ nano export_shell.c
pavani@DESKTOP-380LEIN:~$ gcc export_shell.c -o export_shell
pavani@DESKTOP-380LEIN:~$ ./export_shell
shell> export NAME=Pavani
Exported NAME=Pavani
shell> print NAME
Pavani
shell> shell> export COLLEGE=KLU
sheUnknown command
shell> ll> print COLLEGE^[[3~^[[3~^[[3~^[[3~^[[3~^[[3~^Z
[1]+ Stopped ./export_shell
pavani@DESKTOP-380LEIN:~$ ./export_shell
shell> export NAME=Pavani
Exported NAME=Pavani
shell> print NAME
Pavani
shell> export COLLEGE = KLU
export: invalid variable name
shell> export COLLEGE=KLU
Exported COLLEGE=KLU
shell> print COLLEGE
KLU
shell> exit
pavani@DESKTOP-380LEIN:~$
```

Report:

The export command was implemented to create and update environment variables using `setenv()`.

Variable names are validated before storing their values.

The variables were tested using `getenv()` to verify that they were correctly stored and accessible.